CENTRO UNIVERSITARIO TECNOLÓGICO

91DESARROLLO DE APLICACIONES DE VANGUARDIA 2026 Q1

SECCION 91

ASIGNACION:

PROYECTO/Avance 2

ELABORADO POR:

Anton Leonardo Acosta Romero

DOCENTE

Elvin Osman Bobadilla Salinas

Sistema Inteligente de Precios Dinámicos

- ## Introducción

En un entorno comercial cada vez más competitivo, las micro y pequeñas empresas (pymes) enfrentan dificultades para competir con grandes corporaciones que utilizan algoritmos sofisticados para fijar precios en tiempo real.

Este proyecto propone el desarrollo de un Sistema Inteligente de Precios Dinámicos, una aplicación web cliente-servidor que analiza datos del negocio (ventas históricas, stock, horarios y comportamiento del cliente) para recomendar automáticamente ajustes de precio.

El sistema automatiza la toma de decisiones y permite simular escenarios de ganancia, democratizando el acceso a la inteligencia de precios para optimizar ingresos en pymes.

- ## Información General del Proyecto

Nombre del proyecto: SmartPrice Pyme – Sistema de Recomendación de Precios Dinámicos

JUSTIFICACION: Automatizar la recomendación de precios basada en datos para mejorar la rentabilidad de las pymes.

USUARIOS OBJETIVOS: Dueños de pymes, administradores y vendedores sin necesidad de conocimientos técnicos.

- ## Beneficios del Proyecto

- Incremento de ingresos mediante precios optimizados.
- Decisiones basadas en datos reales.
- Ahorro de tiempo en análisis comerciales.
- Simulación de escenarios de ganancia.
- Escalabilidad a distintos rubros comerciales.

- ## Análisis Técnico

Tecnologías:
Backend: Node.js con Express
Frontend: HTML5, CSS3 y JavaScript
Base de datos: MongoDB
Control de versiones: Git y GitHub

Plataforma: Aplicación web.

Procesos: Autenticación, gestión de productos, registro de ventas y algoritmo de recomendación.

Políticas de respaldo: Backups diarios automatizados y almacenamiento en la nube.

- ## Sistema

Aplicación web cliente-servidor.

Propuesta Comercial Seleccionada

Sistema: SmartPrice Pyme – Sistema Inteligente de Precios Dinámicos

- ## Objetivos del Sistema

Objetivo general: Desarrollar una aplicación web que optimice ingresos mediante precios dinámicos.
Objetivos específicos:
- Implementar algoritmo de análisis de ventas.
- Crear módulo de simulación de ganancias.
- Diseñar interfaz intuitiva.

- ## Alcance del Sistema

Sistema web funcional, gestión de productos, registro de ventas, motor de recomendación, módulo de simulación, documentación técnica y presentación final.

- ## Plan de Trabajo

Modelo de Requerimientos Funcionales

Actividad	**Semana 1**	**Semana 2**	**Semana 3**	**Semana 4**	**Semana 5**	**Semana 6**
Análisis y levantamiento de requisitos	✓	✓				
Diseño de base de datos		✓	✓			
Desarrollo Backend (Node/MongoDB)			✓	✓		
Desarrollo Frontend				✓	✓	
Implementación algoritmo de precios				X	X	
Pruebas y validación					X	X
Documentación y presentación final				X	X	X

Código	Requerimiento	Descripción	Entradas	Salidas
RF-01	Gestión de usuarios	Permite registrar y autenticar usuarios	Nombre, email, rol, contraseña	Perfil de usuario, sesión activa
RF-02	Gestión de productos	CRUD completo de productos	Nombre, SKU, precio, stock	Listado actualizado
RF-03	Registro de ventas	Almacena ventas realizadas	Producto, cantidad, fecha	Historial de ventas
RF-04	Recomendación de precios	Analiza datos y sugiere precio	Stock, ventas recientes	Subir/Bajar/Mantener precio
RF-05	Simulación de ganancias	Calcula proyección de ingresos	Precio simulado, demanda	Gráficos y proyecciones

- **DISEÑO**

```
smartprice-pyme/
│
├── app.js                  # Archivo principal (el que me compartiste)
├── package.json            # Dependencias del proyecto
├── .env                    # Variables de entorno (configuración sensible)
├── .gitignore              # Archivos que no subirás a GitHub
│
├── models/                 # Modelos de datos (los que ya creaste)
│   ├── usuario.js
│   ├── productos.js
│   ├── ventas.js
│   └── recomendaciones.js
│
├── routes/                 # Rutas de la API (endpoints)
│   ├── authRoutes.js           # RF-01: Gestión de usuarios (login, registro)
│   ├── productoRoutes.js       # RF-02: CRUD de productos
│   ├── ventaRoutes.js          # RF-03: Registro de ventas
│   └── recomendacionRoutes.js  # RF-04 y RF-05: Algoritmo y simulación
│
├── controllers/            # Lógica de negocio
│   ├── authController.js
│   ├── productoController.js
│   ├── ventaController.js
│   └── recomendacionController.js # Aquí irá el algoritmo de precios
│
├── middleware/             # Funciones intermedias
```

```
│   └── authMiddleware.js        # Verificar si el usuario está autenticado
│
├── utils/                       # Utilidades y funciones auxiliares
│   └── algoritmoPrecios.js      # El corazón del RF-04
│
├── frontend/                    # Archivos del cliente (HTML, CSS, JS)
│   ├── index.html               # Dashboard principal
│   ├── login.html               # Pantalla de login (RF-01)
│   ├── productos.html           # Gestión de productos (RF-02)
│   ├── ventas.html              # Registro de ventas (RF-03)
│   ├── simulacion.html          # Simulación de ganancias (RF-05)
│   ├── recomendaciones.html     #ver recomendaciones
│   ├── css/
│   │   └── styles.css           # Estilos de toda la app
│   └── js/
│       ├── auth.js              # Lógica de login en el frontend
│       ├── productos.js         # Llamadas a API de productos
│       ├── ventas.js            # Llamadas a API de ventas
│       ├── recomendaciones.js
│       ├──simulacion.js
│       ├── config.js
│       └── dashboard.js         # Mostrar recomendaciones
│
└── uploads/                     # (Opcional) Si necesitas subir imágenes
```

- **DATA**

Requerimiento	Modelo(s) involucrados
RF-01: Gestión de usuarios	Usuario.js
RF-02: Gestión de productos	producto.js
RF-03: Registro de ventas	ventas.js (relaciona producto y usuario)
RF-04: Recomendación de precios	recomendaciones.js (resultado del algoritmo)
RF-05: Simulación de ganancias	recomendaciones.js ((array de simulaciones)

Requerimiento	Endpoints principales
RF-01: Gestion de usuarios	POST /api/auth/registro, POST /api/auth/login, GET /api/auth/perfil
RF-02: Gestion de productos	GET /api/productos, POST /api/productos/crear, PUT /api/productos/editar/:id
RF-03: Registro de ventas	GET /api/ventas, POST /api/ventas/crear, GET /api/ventas/reporte/diario
RF-04: Recomendación de precios	POST /api/recomendaciones/generar/:id, GET /api/recomendaciones
RF-05: Simulación de ganancias	POST /api/recomendaciones/simular/:id, GET /api/recomendaciones/simulaciones/:id

Función	Requerimiento	Descripción
Listar()	RF-04	Lista todas las recomendaciones activas
porProducto()	RF-04	Obtiene recomendación de un producto específico
generarRecomendaciones()	RF-04	Genera nueva recomendación usando el algoritmo
generarTodas()	RF-04	Genera recomendaciones para todos los productos
aplicarRecomendacion()	RF-04	Aplica el cambio de precio sugerido
simularPrecio()	RF-05	Simula ingresos con un precio hipotético
verSimulaciones()	RF-05	Muestra historial de simulaciones
resumenDashboard()	-	Datos resumidos para el dashboard principal
estadisticasRecomendaciones()	-	Estadísticas agregadas de recomendaciones

Archivos frontend	Descripción
css/styles.css	Estilos globales de la aplicación
js/config.js	Configuración de API y funciones comunes
login.html + auth.js	Autenticación de usuarios
index.html + dashboard.js	Dashboard principal con estadísticas
productos.html + productos.js	CRUD completo de productos
ventas.html + ventas.js	Registro de ventas y reportes
recomendaciones.html + recomendaciones.js	Visualización de recomendaciones
simulacion.html + simulacion.js	Simulador de precios

- **LIBRERIAS**

npm install express mongoose cors dotenv bcryptjs jsonwebtoken

Librerías	
express	Framework para crear el servidor web y las rutas
mongoose	Conectar y modelar datos con MongoDB (ya lo tienes)
cors	Permite que tu frontend (HTML/JS) se comunique con el backend
dotenv	Manejar variables de entorno (archivo .env

Librerías	
bcryptjs	Encriptar contraseñas de usuarios (seguridad)
jsonwebtoken	Crear tokens para autenticación (login seguro)

npm install -D nodemon

Librerías	
nodemon	Reinicia automáticamente el servidor cuando guardas cambios

- **ILUSTRACIONES**

Archivo Editar Selección Ver Ir Ejecutar ··· PROYECTO VANGUARD

EXPLORADOR

PROYE...

- CODEVS
  - models
    - productos.js
    - recomendaciones.js
    - usuario.js
    - ventas.js
  - node_modules
  - APPMONGO.JS
  - package-lock.json
  - package.json
- PRESENTACIONES

APPMONGO.JS | ventas.js

CODEVS > APPMONGO.JS > ...

```
4
5
6   //CONECCION A LA BD
7   const{default: mongoose} = require('mongoose');
8   const mongosee = require ('mongoose');
9
10  const url='mongodb://127.0.0.1:27017/BDSMARTPICE1';
11
12  mongoose.connect(url)
13  .then(()=>console.log('CONECTADO A LA BD'))
14  .catch((e)=>console.log('error:'+ e))
15
16
17  //COLLECCION MONGOOSE
18  const Usuario = require('./models/usuario');
19  const Productos = require('./models/productos');
20  const Ventas = require('./models/ventas');
21  const Recomendaciones = require('./models/recomendaciones');
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

```
C:\Users\acost\Desktop\PROYECTO VANGUARDIA\CODEVS>
C:\Users\acost\Desktop\PROYECTO VANGUARDIA\CODEVS>
```

Archivo Editar Selección Ver Ir Ejecutar ··· PROYECTO VANGUARDIA

EXPLORADOR

PROYECTO...

- CODEVS
  - controllers
  - frontend
    - css
    - js
      - auth.js
      - config.js
      - dashboard.js
      - productos.js
      - recomendaciones.js
      - simulacion.js
      - ventas.js
    - index.html
    - login.html
    - productos.html
    - recomendaciones.html
    - simulacion.html
    - ventas.html
  - middleware
  - models
  - node_modules
  - routes
  - uploads
  - utils
  - .env
  - .gitignore
  - app.js
  - package-lock.json
  - package.json
  - PRESENTACIONES
- ESQUEMA
- LÍNEA DE TIEMPO

app.js | authController.js | productoController.js | ventaController.js | recomendacionContro

CODEVS > app.js > ...

```
26  //////////////
27
28  // app.js
29  const express = require('express');
30  const mongoose = require('mongoose');
31  const cors = require('cors');  4.5k (gzipped: 1.9k)
32  const path = require('path');
33  require('dotenv').config();  7.9k (gzipped: 3.4k)
34
35  // Importar rutas
36  const authRoutes = require('./routes/authRoutes');
37  const productoRoutes = require('./routes/productoRoutes');
38  const ventaRoutes = require('./routes/ventaRoutes');
39  const recomendacionRoutes = require('./routes/recomendacionRoutes');
40
41  const app = express();
42  const PORT = process.env.PORT || 3000;
43
44  // Middlewares
45  app.use(cors()); // Permite peticiones del frontend
46  app.use(express.json()); // Para parsear JSON
47  app.use(express.urlencoded({ extended: true })); // Para formularios
48  app.use(express.static(path.join(__dirname, 'frontend'))); // Sirve archivos estáticos
49
50  // Conexión a MongoDB
51  const url = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/BDSMARTPICE2';
52  mongoose.connect(url)
53    .then(() => console.log('✅ CONECTADO A LA BD'))
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

```
PS C:\Users\acost\Desktop\PROYECTO VANGUARDIA\CODEVS> npm run dev
[nodemon] starting `node app.js`
[dotenv@17.3.1] injecting env (5) from .env -- tip: ⚡ secrets for agents: https://dotenvx.com/as2
🚀 Servidor corriendo en http://localhost:3000
📡 API disponible en http://localhost:3000/api
✅ CONECTADO A LA BD
```